claude-windows / bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f\subagents\workflows\wf_b8703ccc-78b\agent-ad662fa621b93bd20.jsonl`
- SHA-256: `7c2b0854b624a7e2b34138eac3111bdd622d09b85b7adbe2ff46a6844a37f5b2`
- Source modified: `2026-06-17T14:34:22+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_b8703ccc-78b`
- session_id: `bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f`

Transcript

1. user (2026-06-17T14:33:01.596Z)

Review the diff of undefined/tools/generate_highlights.py (run `git -C undefined diff origin/master...HEAD`). LENS: --detect-from FEATURE CORRECTNESS. Verify: detection runs on the proxy (video_id keyed off detect_src), the reel is stitched from the full-res --video, the reel is named after --video, the i-1 batch index pairing is correct (enumerate starts at 1), _resolve_detect_from validation (1:1 count, requires explicit --video), and the missing-source check covers both src and detect_src. Find edge cases: staged-file basename collisions across a batch, detect_from given but a path missing, interaction with --skip-ai-handoff. Read the file + tests.

2. assistant (2026-06-17T14:33:02.968Z)

I'll start by getting the git diff to understand what changes are being made.

3. user (2026-06-17T14:33:03.723Z)

(Bash completed with no output)

4. assistant (2026-06-17T14:33:05.240Z)

Let me check the actual git status and find the file more carefully.

5. user (2026-06-17T14:33:05.735Z)

```
On branch docs/golden-regression-fixtures-plan
Your branch and 'origin/docs/golden-regression-fixtures-plan' have diverged,
and have 18 and 1 different commits each, respectively.
  (use "git pull" if you want to integrate the remote branch with yours)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        artifacts/
        docs/KHELSUTRA_PRODUCT_OVERVIEW.md

nothing added to commit but untracked files present (use "git add" to track)
```

6. assistant (2026-06-17T14:33:07.040Z)

Let me find the generate_highlights.py file:

7. user (2026-06-17T14:33:07.116Z)

```
tools\generate_highlights.py
```

8. assistant (2026-06-17T14:33:08.519Z)

Now let me check the git diff for this file:

9. user (2026-06-17T14:33:08.783Z)

```
f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)
8c68143 feat(eval): R1 ? net over-seg promotion guard unblocks the cap6+R4 milestone (#201)
2eaccd5 feat(eval): nightly rally-quality regression tripwire (golden corpus, CPU) (#202)
34b9caa feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding
(#200)
7e8f88f feat(eval): R2 ? task-faithful tolerance-match rally metric (augments tIoU) (#199)
cf8973e docs(eval): R2 design ? task-faithful tolerance-match rally metric (FOR REVIEW) (#198)
369e90c docs(eval): golden-labeled video registry (the growing golden-finalized set) (#197)
c483346 docs(quality): real-footage validation capstone + first ground-truth at-par + dual-eval-
set discipline (#196)
bb4d6e2 feat(eval): Tier 1 (W1.5) ? hard-cap fallback enforces the window cap on continuously-
active trajectories (#195)
7d84319 fix(eval): golden-fixtures audit hardening ? close source_note/label leak + 16 confirmed
audit fixes (#194)
085c07b docs(eval): golden-fixtures discoverability ? CODE_MAP §0/§2.3 + GOLDEN_DATA_SHARING
cross-link (DOC) (#193)
8d3e185 feat(eval): golden fixtures P1 ? de-attribution/minimization tool + M2 refusal gate
(#192)
adb88f4 ci(eval): cross-OS determinism guard for golden fixtures + reconcile project tracker
(#191)
950106b fix(fusion): W2 net-crossings gate is do-no-harm ? never empty a non-empty video (#188
follow-up) (#190)
64494ef feat(eval): golden regression fixtures M1 ? video-free CONTROL-only characterization
harness (#189)
a471745 feat(eval): Tier 1 (W2) ? net-crossings rally-state gate on W1 (?F1 +0.019, 6/0) (#188)
1de2817 docs: freeze Golden Regression Fixtures plan + adopt tracked-project-doc template (#186)
a3b209a feat(windowing): Tier 1 (W1) ? bounded windowing fixes the over-merge (?F1 +0.139, 6/0)
(#187)
90ab087 feat(eval): Tier 0 (E1) ? over-segmentation-aware rally-seg eval + frozen baseline
(#185)
960c00d docs: rally-quality research + eval-framework roadmap (foundation doc) (#184)
```

10. assistant (2026-06-17T14:33:09.956Z)

Let me see what branch I'm on and what the diff looks like:

11. user (2026-06-17T14:33:10.508Z)

(Bash completed with no output)

12. assistant (2026-06-17T14:33:12.094Z)

Let me just read the file directly to understand its structure:

13. user (2026-06-17T14:33:12.218Z)

```
1       """Batch highlight-reel runner ? orchestrates "generate highlights for these videos
2       into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.
3
4       Why this exists (lessons paid for the hard way):
5       - **Never delete other work.** Reels + per-video run.logs are written to a LOCAL work
dir
6         first, then COPIED into the destination (atomically: temp name -> os.replace). The
7         destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a
8         Google-Drive-synced folder once lost finished reels + logs.)
9       - **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync
10        client (deleted files resurrect, partial writes sync). Generate locally; publish once.
11      - **Idempotent re-runs.** Each video's prior DB segments are cleared before processing
```

```
12        (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates
13        duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
14      - **Resumable & concurrency-friendly.** A video whose reel already exists in the
15        destination is skipped (unless --force); launching a second request never undoes the
16        first's published reels. The Gemini path also isolates its chunk workdir per video
17        (PR for output/chunks/<video_id>) so concurrent videos don't collide.
18
19      Usage:
20        python -m tools.generate_highlights --videos-dir "G:\\...\\June 12 2026" \
21            --out "G:\\...\\June 12 2026\\Highlights" --segmenter gemini [--force]
22        python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
23      """
24      from __future__ import annotations
25
26      import argparse
27      import logging
28      import os
29      import shutil
30      import sys
31      import traceback
32      from typing import Any, Dict, List, Optional, cast
33
34      from dotenv import load_dotenv
35
36      sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
37
38      from backend.config import load_config
39      from backend.infrastructure.database import Database
40      from backend.pipeline.segmenters.base import segmenter_registry
41      from backend.pipeline.stitcher.compiler import VideoCompiler
42      from backend.results import Failure
43      from backend.utils.hashing import compute_video_id
44
45      logger = logging.getLogger("highlights_runner")
46
47      # Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.
48      _WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
49      _VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")
50
51
52      def _atomic_publish(local_path: str, dest_path: str) -> None:
53          """Copy local_path -> dest_path so the destination never sees a partial file.
54
55          Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
56          anything else in the destination folder.
57          """
58          os.makedirs(os.path.dirname(dest_path), exist_ok=True)
59          tmp = dest_path + ".publishing.tmp"
60          shutil.copy2(local_path, tmp)
61          os.replace(tmp, dest_path)
62
63
64      def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
65                                    local_work_dir: str, db_path: str, force: bool = False,
66                                    skip_ai_handoff: bool = False, wasb_stream: bool = False)
-> dict:
67          """Generate one highlight reel per input video into out_dir. Returns a summary dict.
68
69          Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
70
71          skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini
handoff);
72              candidate windows become the rallies. wasb_stream: have the WASB detector decode
the
73              video on the fly instead of dumping ~46k PNGs (output-preserving fast path,
```

```
#178).
74          """
75          os.makedirs(out_dir, exist_ok=True)
76          os.makedirs(local_work_dir, exist_ok=True)
77          needs_local = segmenter in _WSL_DETECTORS
78
79          # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is
invoked
80          # standalone ? backend.main does this, but this module is its own entrypoint.
Without it the
81          # AI segmenter finds no API key and silently produces zero rallies. The fully-local
path
82          # (skip_ai_handoff) needs no key, so a missing .env is harmless there.
83          load_dotenv()
84
85          cfg = load_config()
86          # Per-run overrides (typed config; the segmenter reads these via its dict-compat
.get).
87          # Logged by the segmenter itself once logging is configured below.
88          if skip_ai_handoff:
89              cfg.indexing.skip_ai_handoff = True
90          if wasb_stream:
91              cfg.indexing.wasb.stream_video = True
92          db = Database(db_path)
93          seg_cls = segmenter_registry.get(segmenter)
94          if seg_cls is None:
95              raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")
96
97          # Combined batch log (local; published at the end).
98          fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s",
"%H:%M:%S")
99          if not logging.getLogger().handlers:
100             logging.basicConfig(level=logging.INFO,
handlers=[logging.StreamHandler(sys.stdout)])
101             for h in logging.getLogger().handlers:
102                 h.setFormatter(fmt)
103         batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
104         batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
105         batch_fh.setFormatter(fmt)
106         logging.getLogger().addHandler(batch_fh)
107
108         summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [],
"failed": []}
109         logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
110                     len(video_paths), out_dir, segmenter, force)
111
112         for i, src in enumerate(video_paths, 1):
113             name = os.path.basename(src)
114             stem = os.path.splitext(name)[0]
115             reel_name = f"{stem} - Highlights.mp4"
116             dest_reel = os.path.join(out_dir, reel_name)
117             local_reel = os.path.join(local_work_dir, reel_name)
118
119             if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
120                 logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
121                 summary["skipped"].append(name)
122                 continue
123             if not os.path.exists(src):
124                 logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths),
name)
125                 summary["failed"].append(name)
126                 continue
127
```

```
            run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
            run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
            run_fh.setFormatter(fmt)
            logging.getLogger().addHandler(run_fh)
            staged_src: Optional[str] = None
            try:
                logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
                if needs_local:
                    staged_src = os.path.join(local_work_dir, name)
                    if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
                        logger.info("localizing source (%.0f MB)...", os.path.getsize(src) /
1024 / 1024)
                        shutil.copy2(src, staged_src)
                    proc_src = staged_src
                else:
                    proc_src = src

                vid = compute_video_id(proc_src)
                db.clear_video_data(vid)          # idempotent: no cross-run accumulation
                db.add_video(vid, proc_src, "processed", [])
                logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

                # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
                # main.py); cast keeps the type checker honest about that contract.
                result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
                if isinstance(result, Failure):
                    logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
                    summary["failed"].append(name)
                    continue

                segs = db.query_play_segments(vid, {})
                pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
                logger.info("rallies detected: %d", len(pairs))
                if not pairs:
                    logger.warning("no rallies -> no reel")
                    summary["no_rallies"].append(name)
                    continue

                # Compile locally, then publish atomically into the destination.
                VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
                _atomic_publish(local_reel, dest_reel)
                try:
                    os.remove(local_reel)
                except OSError:
                    pass
                logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
                summary["done"].append(name)
            except Exception:
                logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
                summary["failed"].append(name)
            finally:
                logging.getLogger().removeHandler(run_fh)
                run_fh.close()
                # publish this video's run.log (additive; never deletes anything in out_dir)
                try:
                    _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} -
run.log"))
                except OSError:
                    pass
                if staged_src and os.path.exists(staged_src):
                    try:
                        os.remove(staged_src)
                    except OSError:
```

```
189                         pass
190
191         logging.getLogger().removeHandler(batch_fh)
192         batch_fh.close()
193         try:
194             _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
195         except OSError:
196             pass
197         logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
198         return summary
199
200
201     def _resolve_videos(args) -> List[str]:
202         if args.video:
203             return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v
in args.video]
204         if args.videos_dir:
205             return sorted(
206                 os.path.join(args.videos_dir, f) for f in os.listdir(args.videos_dir)
207                 if f.endswith(_VIDEO_EXTS) and os.path.isfile(os.path.join(args.videos_dir,
f))
208             )
209         raise SystemExit("provide --videos-dir or one/more --video")
210
211
212     def main() -> None:
213         ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
214         ap.add_argument("--videos-dir", help="folder of source videos (all videos in it,
unless --video given)")
215         ap.add_argument("--video", action="append", help="explicit source video
(repeatable)")
216         ap.add_argument("--out", required=True, help="destination folder for reels +
run.logs")
217         ap.add_argument("--segmenter", default="gemini", help="segmenter registry key
(default: gemini)")
218         ap.add_argument("--local-work", default=os.path.join("output", "highlights_work"),
219                         help="local staging dir (reels compiled here, then published)")
220         ap.add_argument("--db", default=os.path.join("output", "sports_indexer.db"))
221         ap.add_argument("--force", action="store_true", help="regenerate even if a reel
already exists")
222         ap.add_argument("--skip-ai-handoff", action="store_true",
223                         help="FULLY-LOCAL: trajectory-hybrid segmenters emit candidate
windows as "
224                              "rallies with NO Gemini handoff (no API key / upload). Ignored
by --segmenter gemini.")
225         ap.add_argument("--wasb-stream", action="store_true",
226                         help="WASB decode-on-the-fly fast path (#178; output-preserving),
instead of "
227                              "dumping ~46k PNGs. Only affects wasb_hybrid.")
228         args = ap.parse_args()
229         generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
230                                   local_work_dir=args.local_work, db_path=args.db,
force=args.force,
231                                   skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream)
232
233
234     if __name__ == "__main__":
235         main()
236
```

14. assistant (2026-06-17T14:33:13.777Z)

Now let me look for tests related to this file:

15. user (2026-06-17T14:33:14.207Z)

```
No files found
```

16. assistant (2026-06-17T14:33:15.718Z)

Let me search for any test files that might test this module:

17. user (2026-06-17T14:33:16.600Z)

```
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/run_all_tests.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/scratch/test_ai_segmenter.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/scratch/test_thresholds.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_ablation.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_annotation_providers.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_api_endpoints.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_async_jobs.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_audio_features.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_automated_provider.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_base.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_calibrate_local.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_calibrate_wasb.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_calibration.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_cleanup_caches.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_compiler.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_config.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_config_wasb.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_database.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_detector_keying.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_distill_local.py
```

18. assistant (2026-06-17T14:33:18.092Z)

Let me check the actual git log to see recent changes:

19. user (2026-06-17T14:33:18.775Z)

```
commit b6a39c469bc4b72069c6d9707b037022a658c1d0
Author: Avi <avi.dullu@gmail.com>
Date:   Tue Jun 16 00:43:21 2026 +0530

    feat(runner): fully-local no-AI segmenter mode (+ WASB stream flag, .env load) (#183)

    Adds a "fully local, no cloud AI" path to the trajectory-hybrid segmenters and the batch
```

```
    runner ? for benchmarking the local detector against the Gemini path and for running with
    zero cloud dependency.

    - IndexingConfig.skip_ai_handoff (default OFF): when set, TrajectoryHybridSegmenter
      (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skips the Phase-3 AI handoff and emits its
      local candidate windows AS the rallies ? no provider, no API key, nothing uploaded.
      FusionSegmenter still applies its Gate-A/B window selection via _select_for_handoff.
    - tools/generate_highlights.py: --skip-ai-handoff and --wasb-stream flags inject the typed
      config overrides (skip_ai_handoff, indexing.wasb.stream_video). The runner now also
      load_dotenv()s so GEMINI_API_KEY is available when it is invoked standalone ? previously
      only backend.main loaded .env, so a runner launched without the key exported silently
      produced zero rallies on the Gemini path. The local path needs no key.
    - Tests: skip-AI emits candidate windows as rallies with no provider/key/clip upload and a
      local detector tag; config default-off + file override; runner flag wiring.

    Full suite 713 passed / 7 skipped; ruff + mypy clean (CI red account-wide on the billing
    outage; merging on verified local green per the standing working agreement).

    Co-authored-by: Claude Opus 4.8 (1M context) <noreply@anthropic.com>

diff --git a/tools/generate_highlights.py b/tools/generate_highlights.py
index 949bda1..ae9f045 100644
--- a/tools/generate_highlights.py
+++ b/tools/generate_highlights.py
@@ -31,6 +31,8 @@ import sys
 import traceback
 from typing import Any, Dict, List, Optional, cast

+from dotenv import load_dotenv
+
 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

 from backend.config import load_config
@@ -60,16 +62,33 @@ def _atomic_publish(local_path: str, dest_path: str) -> None:


 def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
-                              local_work_dir: str, db_path: str, force: bool = False) -> dict:
+                              local_work_dir: str, db_path: str, force: bool = False,
+                              skip_ai_handoff: bool = False, wasb_stream: bool = False) ->
dict:
     """Generate one highlight reel per input video into out_dir. Returns a summary dict.

     Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
+
+    skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini handoff);
+        candidate windows become the rallies. wasb_stream: have the WASB detector decode the
+        video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).
     """
     os.makedirs(out_dir, exist_ok=True)
     os.makedirs(local_work_dir, exist_ok=True)
     needs_local = segmenter in _WSL_DETECTORS

+    # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is invoked
+    # standalone ? backend.main does this, but this module is its own entrypoint. Without it
the
+    # AI segmenter finds no API key and silently produces zero rallies. The fully-local path
+    # (skip_ai_handoff) needs no key, so a missing .env is harmless there.
+    load_dotenv()
+
     cfg = load_config()
+    # Per-run overrides (typed config; the segmenter reads these via its dict-compat .get).
+    # Logged by the segmenter itself once logging is configured below.
+    if skip_ai_handoff:
+        cfg.indexing.skip_ai_handoff = True
```

```
+    if wasb_stream:
+        cfg.indexing.wasb.stream_video = True
     db = Database(db_path)
     seg_cls = segmenter_registry.get(segmenter)
     if seg_cls is None:
@@ -200,9 +219,16 @@ def main() -> None:
                     help="local staging dir (reels compiled here, then published)")
     ap.add_argument("--db", default=os.path.join("output", "sports_indexer.db"))
     ap.add_argument("--force", action="store_true", help="regenerate even if a reel already
exists")
+    ap.add_argument("--skip-ai-handoff", action="store_true",
+                    help="FULLY-LOCAL: trajectory-hybrid segmenters emit candidate windows as "
+                         "rallies with NO Gemini handoff (no API key / upload). Ignored by
--segmenter gemini.")
+    ap.add_argument("--wasb-stream", action="store_true",
+                    help="WASB decode-on-the-fly fast path (#178; output-preserving), instead
of "
+                         "dumping ~46k PNGs. Only affects wasb_hybrid.")
     args = ap.parse_args()
     generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
-                              local_work_dir=args.local_work, db_path=args.db,
force=args.force)
+                              local_work_dir=args.local_work, db_path=args.db,
force=args.force,
+                              skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream)


 if __name__ == "__main__":

commit 4942d51ddcb5a6627741f049b03a4c0aaa8b5ec6
Author: Avi <avi.dullu@gmail.com>
Date:   Mon Jun 15 23:01:02 2026 +0530

    feat(tools): batch highlight RUNNER ? clean, non-destructive, Drive-safe reel generation
(#182)

    The orchestrated way to do "make reels for these videos into this folder", paid for by
    two real incidents: a hand-rolled loop that (a) wrote reels directly into a Google-Drive
    folder (sync races resurrected deleted files / synced partials) and (b) bulk-`rm`'d the
    folder to clear stale output, losing finished reels + run.logs.

    tools/generate_highlights.py guarantees:
    - Generate LOCALLY, then publish atomically (copy -> os.replace) into the destination ?
      Drive-sync-safe; the dest is only ever ADDED to, NEVER bulk-deleted.
    - Idempotent: clear_video_data per video before processing (add_video is UPSERT and
      doesn't clear children), so re-runs never accumulate duplicate rallies (pairs with the
      compiler overlap-merge #180).
    - Resumable: skip published reels unless --force; launching a second request never undoes
      the first's published work.
    - Per-video `<stem> - run.log` + combined `_batch_run.log` published additively.
    - WSL detectors localize the source first (Drive invisible to /mnt).

    Tests: atomic-publish leaves siblings intact + no .tmp; _resolve_videos (dir/explicit);
    skip-existing is non-destructive (other reels untouched, segmenter not run). CODE_MAP §2.4
    documents it as THE batch-reel entrypoint. ruff+mypy clean; full suite 696 passed.

    Co-authored-by: Claude Fable 5 <noreply@anthropic.com>

diff --git a/tools/generate_highlights.py b/tools/generate_highlights.py
new file mode 100644
index 0000000..949bda1
--- /dev/null
+++ b/tools/generate_highlights.py
@@ -0,0 +1,209 @@
```

```
+"""Batch highlight-reel runner ? orchestrates "generate highlights for these videos
+into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.
+
+Why this exists (lessons paid for the hard way):
+- **Never delete other work.** Reels + per-video run.logs are written to a LOCAL work dir
+  first, then COPIED into the destination (atomically: temp name -> os.replace). The
+  destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a
+  Google-Drive-synced folder once lost finished reels + logs.)
+- **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync
+  client (deleted files resurrect, partial writes sync). Generate locally; publish once.
+- **Idempotent re-runs.** Each video's prior DB segments are cleared before processing
+  (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates
+  duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
+- **Resumable & concurrency-friendly.** A video whose reel already exists in the
+  destination is skipped (unless --force); launching a second request never undoes the
+  first's published reels. The Gemini path also isolates its chunk workdir per video
+  (PR for output/chunks/<video_id>) so concurrent videos don't collide.
+
+Usage:
+  python -m tools.generate_highlights --videos-dir "G:\\...\\June 12 2026" \
+      --out "G:\\...\\June 12 2026\\Highlights" --segmenter gemini [--force]
+  python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
+"""
+from __future__ import annotations
+
+import argparse
+import logging
+import os
+import shutil
+import sys
+import traceback
+from typing import Any, Dict, List, Optional, cast
+
+sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
+
+from backend.config import load_config
+from backend.infrastructure.database import Database
+from backend.pipeline.segmenters.base import segmenter_registry
+from backend.pipeline.stitcher.compiler import VideoCompiler
+from backend.results import Failure
+from backend.utils.hashing import compute_video_id
+
+logger = logging.getLogger("highlights_runner")
+
+# Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.
+_WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
+_VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")
+
+
+def _atomic_publish(local_path: str, dest_path: str) -> None:
+    """Copy local_path -> dest_path so the destination never sees a partial file.
+
+    Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
+    anything else in the destination folder.
+    """
+    os.makedirs(os.path.dirname(dest_path), exist_ok=True)
+    tmp = dest_path + ".publishing.tmp"
+    shutil.copy2(local_path, tmp)
+    os.replace(tmp, dest_path)
+
+
+def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
+                              local_work_dir: str, db_path: str, force: bool = False) -> dict:
+    """Generate one highlight reel per input video into out_dir. Returns a summary dict.
+
```

```
+    Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
+    """
+    os.makedirs(out_dir, exist_ok=True)
+    os.makedirs(local_work_dir, exist_ok=True)
+    needs_local = segmenter in _WSL_DETECTORS
+
+    cfg = load_config()
+    db = Database(db_path)
+    seg_cls = segmenter_registry.get(segmenter)
+    if seg_cls is None:
+        raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")
+
+    # Combined batch log (local; published at the end).
+    fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s", "%H:%M:%S")
+    if not logging.getLogger().handlers:
+        logging.basicConfig(level=logging.INFO, handlers=[logging.StreamHandler(sys.stdout)])
+        for h in logging.getLogger().handlers:
+            h.setFormatter(fmt)
+    batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
+    batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
+    batch_fh.setFormatter(fmt)
+    logging.getLogger().addHandler(batch_fh)
+
+    summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [], "failed": []}
+    logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
+                len(video_paths), out_dir, segmenter, force)
+
+    for i, src in enumerate(video_paths, 1):
+        name = os.path.basename(src)
+        stem = os.path.splitext(name)[0]
+        reel_name = f"{stem} - Highlights.mp4"
+        dest_reel = os.path.join(out_dir, reel_name)
+        local_reel = os.path.join(local_work_dir, reel_name)
+
+        if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
+            logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
+            summary["skipped"].append(name)
+            continue
+        if not os.path.exists(src):
+            logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
+            summary["failed"].append(name)
+            continue
+
+        run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
+        run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
+        run_fh.setFormatter(fmt)
+        logging.getLogger().addHandler(run_fh)
+        staged_src: Optional[str] = None
+        try:
+            logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
+            if needs_local:
+                staged_src = os.path.join(local_work_dir, name)
+                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
+                    logger.info("localizing source (%.0f MB)...", os.path.getsize(src) / 1024 /
1024)
+                    shutil.copy2(src, staged_src)
+                proc_src = staged_src
+            else:
+                proc_src = src
+
+            vid = compute_video_id(proc_src)
+            db.clear_video_data(vid)          # idempotent: no cross-run accumulation
+            db.add_video(vid, proc_src, "processed", [])
```

```
+            logger.info("video_id=%s segmenting (%s)...", vid, segmenter)
+
+            # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
+            # main.py); cast keeps the type checker honest about that contract.
+            result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
+            if isinstance(result, Failure):
+                logger.error("segmentation FAILED: %s", getattr(result, "message", result))
+                summary["failed"].append(name)
+                continue
+
+            segs = db.query_play_segments(vid, {})
+            pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
+            logger.info("rallies detected: %d", len(pairs))
+            if not pairs:
+                logger.warning("no rallies -> no reel")
+                summary["no_rallies"].append(name)
+                continue
+
+            # Compile locally, then publish atomically into the destination.
+            VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
+            _atomic_publish(local_reel, dest_reel)
+            try:
+                os.remove(local_reel)
+            except OSError:
+                pass
+            logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
+            summary["done"].append(name)
+        except Exception:
+            logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
+            summary["failed"].append(name)
+        finally:
+            logging.getLogger().removeHandler(run_fh)
+            run_fh.close()
+            # publish this video's run.log (additive; never deletes anything in out_dir)
+            try:
+                _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} - run.log"))
+            except OSError:
+                pass
+            if staged_src and os.path.exists(staged_src):
+                try:
+                    os.remove(staged_src)
+                except OSError:
+                    pass
+
+    logging.getLogger().removeHandler(batch_fh)
```

20. assistant (2026-06-17T14:33:20.398Z)

Now let me search for related tests more thoroughly:

21. user (2026-06-17T14:33:21.334Z)

(Bash completed with no output)

22. assistant (2026-06-17T14:33:22.527Z)

Let me try a broader search for test files:

23. user (2026-06-17T14:33:22.938Z)

```
total 810
drwxr-xr-x 1 avidu 197609     0 Jun 17 19:44 .
drwxr-xr-x 1 avidu 197609     0 Jun 17 19:44 ..
-rw-r--r-- 1 avidu 197609    16 May 30 14:08 __init__.py
```

```
drwxr-xr-x 1 avidu 197609     0 Jun 15 22:58 __pycache__
-rw-r--r-- 1 avidu 197609 23942 Jun 14 15:59 test_ablation.py
-rw-r--r-- 1 avidu 197609  3182 May 30 21:58 test_annotation_providers.py
-rw-r--r-- 1 avidu 197609 14147 Jun 15 14:11 test_api_endpoints.py
-rw-r--r-- 1 avidu 197609 15194 Jun 15 14:11 test_async_jobs.py
-rw-r--r-- 1 avidu 197609  3373 Jun 14 02:06 test_audio_features.py
-rw-r--r-- 1 avidu 197609  3320 Jun  7 14:14 test_automated_provider.py
-rw-r--r-- 1 avidu 197609  8211 Jun 15 14:11 test_base.py
-rw-r--r-- 1 avidu 197609  1122 Jun  6 23:03 test_calibrate_local.py
-rw-r--r-- 1 avidu 197609  2032 Jun  6 18:05 test_calibrate_wasb.py
-rw-r--r-- 1 avidu 197609  3668 Jun  6 16:49 test_calibration.py
-rw-r--r-- 1 avidu 197609  3609 Jun 13 21:02 test_cleanup_caches.py
-rw-r--r-- 1 avidu 197609  4131 Jun 15 22:37 test_compiler.py
-rw-r--r-- 1 avidu 197609 13445 Jun 16 01:51 test_config.py
-rw-r--r-- 1 avidu 197609  2645 Jun 16 00:18 test_config_wasb.py
-rw-r--r-- 1 avidu 197609 14853 Jun 14 16:28 test_database.py
-rw-r--r-- 1 avidu 197609  3588 Jun 10 13:14 test_detector_keying.py
-rw-r--r-- 1 avidu 197609  1498 Jun  6 23:03 test_distill_local.py
-rw-r--r-- 1 avidu 197609  2602 Jun 10 15:37 test_eval_annotations.py
-rw-r--r-- 1 avidu 197609  2464 May 30 14:08 test_eval_batch.py
-rw-r--r-- 1 avidu 197609  2124 Jun 11 10:48 test_eval_classifier.py
-rw-r--r-- 1 avidu 197609  5122 Jun  7 14:14 test_eval_harness.py
-rw-r--r-- 1 avidu 197609  4995 Jun 10 13:45 test_eval_metrics.py
-rw-r--r-- 1 avidu 197609  3689 Jun  8 16:37 test_eval_partial_labels.py
-rw-r--r-- 1 avidu 197609  6619 Jun 11 10:48 test_eval_regression.py
-rw-r--r-- 1 avidu 197609  3086 Jun 14 02:31 test_eval_stats.py
```

24. assistant (2026-06-17T14:33:24.402Z)

Let me search for any tests related to the batch runner or generate_highlights:

25. user (2026-06-17T14:33:25.842Z)

```
C:\Users\avidu\Projects\badminton-highlight-
indexer\tests/test_generate_highlights.py:"""tools/generate_highlights.py ? the batch highlight
runner. The load-bearing guarantees
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_generate_highlights.py:import
tools.generate_highlights as gh
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_generate_highlights.py:
summary = gh.generate_highlights_batch(
C:\Users\avidu\Projects\badminton-highlight-indexer\tests/test_generate_highlights.py:
gh.generate_highlights_batch(
Binary file C:\Users\avidu\Projects\badminton-highlight-
indexer\tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc matches
```

26. assistant (2026-06-17T14:33:27.029Z)

Perfect! Now let me read the test file:

27. user (2026-06-17T14:33:27.420Z)

```
1       """tools/generate_highlights.py ? the batch highlight runner. The load-bearing
guarantees
2       are NON-DESTRUCTIVE + atomic publish + idempotent skip; those are what we lock here
3       (heavy deps ? segmenter/compiler/db ? are mocked)."""
4       import os
5       from unittest.mock import MagicMock, patch
6
7       import tools.generate_highlights as gh
8
9
10      def test_atomic_publish_is_nondestructive(tmp_path):
11          src = tmp_path / "local_reel.mp4"
12          src.write_bytes(b"reel-bytes")
```

```
        dest_dir = tmp_path / "out"
        dest_dir.mkdir()
        sibling = dest_dir / "Someone Else - Highlights.mp4"
        sibling.write_bytes(b"do-not-touch")

        gh._atomic_publish(str(src), str(dest_dir / "Me - Highlights.mp4"))

        assert (dest_dir / "Me - Highlights.mp4").read_bytes() == b"reel-bytes"
        assert sibling.read_bytes() == b"do-not-touch"          # sibling untouched
        assert not list(dest_dir.glob("*.publishing.tmp"))      # no partial left behind


    def test_resolve_videos_explicit_and_dir(tmp_path):
        (tmp_path / "a.MP4").write_bytes(b"x")
        (tmp_path / "b.mp4").write_bytes(b"x")
        (tmp_path / "notes.txt").write_text("ignore")
        args = MagicMock(video=None, videos_dir=str(tmp_path))
        found = gh._resolve_videos(args)
        assert [os.path.basename(p) for p in found] == ["a.MP4", "b.mp4"]   # videos only,
sorted
        args2 = MagicMock(video=["only.MP4"], videos_dir=str(tmp_path))
        assert gh._resolve_videos(args2) == [os.path.join(str(tmp_path), "only.MP4")]


    def test_batch_skips_existing_reel_and_preserves_other_work(tmp_path):
        """force=False: a video whose reel already exists is skipped, the segmenter is never
        called, and unrelated files already in the destination are left intact."""
        out = tmp_path / "out"
        out.mkdir()
        (out / "Adarsh v Avi - Highlights.mp4").write_bytes(b"already-published")
        (out / "Adarsh v Avi - run.log").write_text("prior log")
        other = out / "Someone Else - Highlights.mp4"
        other.write_bytes(b"untouched")
        src = tmp_path / "Adarsh v Avi.MP4"
        src.write_bytes(b"src")

        seg_registry = MagicMock()
        with patch.object(gh, "load_config", return_value={}), \
             patch.object(gh, "Database", return_value=MagicMock()), \
             patch.object(gh, "segmenter_registry", seg_registry):
            summary = gh.generate_highlights_batch(
                [str(src)], str(out), segmenter="gemini",
                local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
force=False)

        assert summary["skipped"] == ["Adarsh v Avi.MP4"]
        # never processed: the resolved segmenter class was never instantiated/run
        seg_registry.get.return_value.assert_not_called()
        assert other.read_bytes() == b"untouched"            # other work preserved
        assert (out / "Adarsh v Avi - Highlights.mp4").read_bytes() == b"already-published"
# not regenerated


    def test_batch_applies_local_overrides(tmp_path):
        """--skip-ai-handoff / --wasb-stream inject into the typed config before
segmentation."""
        from backend.config import AppConfig
        cfg = AppConfig()
        assert cfg.indexing.skip_ai_handoff is False          # default off
        assert cfg.indexing.wasb.stream_video is False

        out = tmp_path / "out"
        out.mkdir()
        src = tmp_path / "clip.MP4"
        src.write_bytes(b"src")
```

```
74
75          seg_instance = MagicMock()
76          seg_instance.process_video.return_value = []          # no rallies -> no
compile/publish
77          seg_registry = MagicMock()
78          seg_registry.get.return_value = MagicMock(return_value=seg_instance)
79
80          with patch.object(gh, "load_config", return_value=cfg), \
81               patch.object(gh, "Database", return_value=MagicMock()), \
82               patch.object(gh, "compute_video_id", return_value="vid123"), \
83               patch.object(gh, "segmenter_registry", seg_registry):
84              gh.generate_highlights_batch(
85                  [str(src)], str(out), segmenter="gemini",
86                  local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
87                  force=True, skip_ai_handoff=True, wasb_stream=True)
88
89          assert cfg.indexing.skip_ai_handoff is True
90          assert cfg.indexing.wasb.stream_video is True
91
```

28. assistant (2026-06-17T14:33:29.266Z)

Now let me check the git log for more context on recent changes and understand what the review task is about:

29. user (2026-06-17T14:33:29.984Z)

(Bash completed with no output)

30. assistant (2026-06-17T14:33:31.456Z)

Let me check what the current branch is tracking:

31. user (2026-06-17T14:33:32.275Z)

```
+ bhi-execpolicy                            5cb7bbf (C:/Users/avidu/Projects/badminton-
highlight-indexer/.claude/worktrees/bhi-execpolicy) [origin/bhi-execpolicy: gone] feat(jobs):
worker self-scheduling drain loop honoring WAIT_AND_RESUME (EXECUTION_POLICY P3a)
* docs/golden-regression-fixtures-plan      f5f5e52 [origin/docs/golden-regression-fixtures-
plan: ahead 18, behind 1] feat(eval): R5 ? blob-fallback splits the continuously-active blob
(default-OFF) (#203)
+ docs/golden-videos-tracker                c33cf1d (C:/Users/avidu/Projects/badminton-
highlight-indexer/.claude/worktrees/docs-golden) [origin/docs/golden-videos-tracker] docs(eval):
golden-labeled video registry (the growing golden-finalized set)
+ docs/quality-iterations-realfootage       ce7eb47 (C:/Users/avidu/Projects/badminton-
highlight-indexer/.claude/worktrees/docs-today) [origin/docs/quality-iterations-realfootage]
docs(quality): real-footage validation capstone + first ground-truth at-par + dual-eval-set
discipline
+ docs/rally-quality-research               877ab1f (C:/Users/avidu/Projects/badminton-
highlight-indexer/.claude/worktrees/docs-rally-quality) [origin/docs/rally-quality-research]
docs: attach project plan + definition of done to the rally-quality doc
  feat/highlights-runner                    bd3c8b5 [origin/feat/highlights-runner] feat(tools):
batch highlight RUNNER ? clean, non-destructive, Drive-safe reel generation
+ feat/local-no-ai-segmenter               3ea7ad4 (C:/Users/avidu/Projects/badminton-
highlight-indexer/.claude/worktrees/local-noai) [origin/feat/local-no-ai-segmenter]
feat(runner): fully-local no-AI segmenter mode (+ WASB stream flag, .env load)
  feat/r1-milestone-config-flip             ef60036 [origin/feat/r1-milestone-config-flip]
feat(config): R1 milestone ? flip windowing defaults to cap=6 + R4 [DO NOT MERGE w/o product-go]
  feat/r1-net-overseg-guard                 0740c66 [origin/feat/r1-net-overseg-guard]
feat(eval): R1 ? net over-seg promotion guard unblocks the cap6+R4 milestone
+ feat/r2-tolerance-metric                  a450c53 (C:/Users/avidu/Projects/badminton-
highlight-indexer/.claude/worktrees/r2-impl) [origin/feat/r2-tolerance-metric] feat(eval): R2 ?
task-faithful tolerance-match rally metric (augments tIoU)
+ feat/r4-inactivity-end                    e248d2d (C:/Users/avidu/Projects/badminton-
```

highlight-indexer/.claude/worktrees/r4-endrule) [origin/feat/r4-inactivity-end] feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding
  feat/r5-blob-splitter   8a45334 [origin/feat/r5-blob-splitter] feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF)
+ feat/tier0-rally-seg-eval   9e03e5a (C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier0-eval) [origin/feat/tier0-rally-seg-eval] feat(eval): Tier 0 (E1) ? over-segmentation-aware rally-seg eval + frozen baseline
+ feat/w1-hard-cap-fallback   614325d (C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hardchop) [origin/feat/w1-hard-cap-fallback] feat(eval): Tier 1 (W1.5) ? hard-cap fallback enforces the window cap on continuously-active trajectories
+ feat/w2-netcross-gate   effd920 (C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-netcross) [origin/feat/w2-netcross-gate] feat(eval): Tier 1 (W2) ? net-crossings rally-state gate on W1 (?F1 +0.019, 6/0)
+ feat/w2-never-empty-safeguard   5ba6955 (C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-safeguard) [origin/feat/w2-never-empty-safeguard] fix(fusion): W2 net-crossings gate is do-no-harm ? never empty a non-empty video (#188 follow-up)
  fix/compiler-dedup-overlapping-segments 1deb917 [origin/fix/compiler-dedup-overlapping-segments] fix(stitcher): merge overlapping/duplicate segments so a reel never stitches the same rally twice
  fix/gemini-per-video-chunk-dir   312cfbc [origin/fix/gemini-per-video-chunk-dir] fix(gemini): isolate chunk workdir per video (output/chunks/<video_id>) ? no cross-run clobber
+ fix/runner-stitch-watermark   a6adcc5 (C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/pr-stitch) [origin/fix/runner-stitch-watermark] fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels
  fix/wasb-test-mocks-readability-preflight 35fb801 [origin/fix/wasb-test-mocks-readability-preflight] fix(tests): update WASB stage/run_predict mocks for the WSL readability preflight (#177 follow-up)
  fix/wsl-unreadable-source-preflight   514fea0 [origin/fix/wsl-unreadable-source-preflight] fix(detectors): fail fast with an actionable error when a video source is unreadable from WSL
+ master   a3b209a (C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing) [origin/master: behind 17] feat(windowing): Tier 1 (W1) ? bounded windowing fixes the over-merge (?F1 +0.139, 6/0) (#187)
+ perf/wasb-fast-frames   7e0881a (C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed) [origin/perf/wasb-fast-frames] fix(wasb): make streaming fast-path output-preserving ? patch read_image (PIL), not cv2.imread
+ run/boxhill17-highlights   34b9caa (C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17) [origin/master: behind 3] feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding (#200)
  worktree-agent-a771ff1d85e18f2ed   46ad601 fix(detectors): fail fast with an actionable error when a video source is unreadable from WSL (#177)

32. assistant (2026-06-17T14:33:34.113Z)

I see the current branch is `fix/runner-stitch-watermark` with commit `a6adcc5`. Let me check the git diff for this feature:

33. user (2026-06-17T14:33:34.660Z)

(Bash completed with no output)

34. assistant (2026-06-17T14:33:36.659Z)

Let me check what changes are in the worktree we're in:

35. user (2026-06-17T14:33:37.221Z)

a6adcc5 fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels
f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)
8c68143 feat(eval): R1 ? net over-seg promotion guard unblocks the cap6+R4 milestone (#201)
2eaccd5 feat(eval): nightly rally-quality regression tripwire (golden corpus, CPU) (#202)
34b9caa feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding (#200)

36. assistant (2026-06-17T14:33:38.845Z)

Perfect! Now let me get the diff for this specific commit:

37. user (2026-06-17T14:33:39.218Z)

```
"""Batch highlight-reel runner ? orchestrates "generate highlights for these videos
into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.

Why this exists (lessons paid for the hard way):
- **Never delete other work.** Reels + per-video run.logs are written to a LOCAL work dir
  first, then COPIED into the destination (atomically: temp name -> os.replace). The
  destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a
  Google-Drive-synced folder once lost finished reels + logs.)
- **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync
  client (deleted files resurrect, partial writes sync). Generate locally; publish once.
- **Idempotent re-runs.** Each video's prior DB segments are cleared before processing
  (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates
  duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
- **Resumable & concurrency-friendly.** A video whose reel already exists in the
  destination is skipped (unless --force); launching a second request never undoes the
  first's published reels. The Gemini path also isolates its chunk workdir per video
  (PR for output/chunks/<video_id>) so concurrent videos don't collide.

Usage:
  python -m tools.generate_highlights --videos-dir "G:\\...\\June 12 2026" \
      --out "G:\\...\\June 12 2026\\Highlights" --segmenter gemini [--force]
  python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
"""
from __future__ import annotations

import argparse
import logging
import os
import shutil
import sys
import traceback
from typing import Any, Dict, List, Optional, cast

from dotenv import load_dotenv

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from backend.config import load_config
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.results import Failure
from backend.utils.hashing import compute_video_id

logger = logging.getLogger("highlights_runner")

# Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.
_WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
_VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")


def _atomic_publish(local_path: str, dest_path: str) -> None:
    """Copy local_path -> dest_path so the destination never sees a partial file.

    Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
    anything else in the destination folder.
    """
    os.makedirs(os.path.dirname(dest_path), exist_ok=True)
```

```
    tmp = dest_path + ".publishing.tmp"
    shutil.copy2(local_path, tmp)
    os.replace(tmp, dest_path)


def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                              local_work_dir: str, db_path: str, force: bool = False,
                              skip_ai_handoff: bool = False, wasb_stream: bool = False,
                              detect_from_paths: Optional[List[str]] = None) -> dict:
    """Generate one highlight reel per input video into out_dir. Returns a summary dict.

    Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.

    skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini handoff);
        candidate windows become the rallies. wasb_stream: have the WASB detector decode the
        video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).
    detect_from_paths: optional list parallel to video_paths. When given, the detector runs on
        detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source) but the
        reel is stitched from video_paths[i] at full resolution. Rally timestamps transfer 1:1
        (a pure temporal-preserving downscale shares the timeline). None = detect+stitch the
        same source (legacy behaviour).
    """
    os.makedirs(out_dir, exist_ok=True)
    os.makedirs(local_work_dir, exist_ok=True)
    needs_local = segmenter in _WSL_DETECTORS

    # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is invoked
    # standalone ? backend.main does this, but this module is its own entrypoint. Without it the
    # AI segmenter finds no API key and silently produces zero rallies. The fully-local path
    # (skip_ai_handoff) needs no key, so a missing .env is harmless there.
    load_dotenv()

    cfg = load_config()
    # Per-run overrides (typed config; the segmenter reads these via its dict-compat .get).
    # Logged by the segmenter itself once logging is configured below.
    if skip_ai_handoff:
        cfg.indexing.skip_ai_handoff = True
    if wasb_stream:
        cfg.indexing.wasb.stream_video = True
    db = Database(db_path)
    seg_cls = segmenter_registry.get(segmenter)
    if seg_cls is None:
        raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")

    # Combined batch log (local; published at the end).
    fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s", "%H:%M:%S")
    if not logging.getLogger().handlers:
        logging.basicConfig(level=logging.INFO, handlers=[logging.StreamHandler(sys.stdout)])
        for h in logging.getLogger().handlers:
            h.setFormatter(fmt)
    batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
    batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
    batch_fh.setFormatter(fmt)
    logging.getLogger().addHandler(batch_fh)

    summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [], "failed": []}
    logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
                len(video_paths), out_dir, segmenter, force)

    for i, src in enumerate(video_paths, 1):
        name = os.path.basename(src)
        stem = os.path.splitext(name)[0]
        reel_name = f"{stem} - Highlights.mp4"
        dest_reel = os.path.join(out_dir, reel_name)
```

```
        local_reel = os.path.join(local_work_dir, reel_name)
        # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
        # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
        detect_src = detect_from_paths[i - 1] if detect_from_paths else src

        if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
            logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
            summary["skipped"].append(name)
            continue
        if not os.path.exists(src) or not os.path.exists(detect_src):
            logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
                           i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
            summary["failed"].append(name)
            continue

        run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
        run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
        run_fh.setFormatter(fmt)
        logging.getLogger().addHandler(run_fh)
        staged_src: Optional[str] = None
        try:
            logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
            # Localize the DETECTION source for WSL detectors (the WSL FS can't read Drive/UNC
            # via /mnt). The stitch source (`src`) is read by ffmpeg directly during compile, so
            # only the detector input ever needs staging.
            if needs_local:
                staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
                    logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
                    shutil.copy2(detect_src, staged_src)
                proc_detect = staged_src
            else:
                proc_detect = detect_src

            vid = compute_video_id(proc_detect)
            db.clear_video_data(vid)          # idempotent: no cross-run accumulation
            db.add_video(vid, proc_detect, "processed", [])
            if detect_src != src:
                logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
                            os.path.basename(detect_src), name)
            logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

            # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
            # main.py); cast keeps the type checker honest about that contract.
            result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_detect)
            if isinstance(result, Failure):
                logger.error("segmentation FAILED: %s", getattr(result, "message", result))
                summary["failed"].append(name)
                continue

            segs = db.query_play_segments(vid, {})
            pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
            logger.info("rallies detected: %d", len(pairs))
            if not pairs:
                logger.warning("no rallies -> no reel")
                summary["no_rallies"].append(name)
                continue

            # Stitch source: the original `src` when detecting on a separate proxy (hi-res
reel);
            # otherwise the (possibly localized) detection input ? identical to legacy
```

```
behaviour.
            stitch_src = src if detect_from_paths else proc_detect
            # Honor the configured branding watermark + segment paddings (stitching.*). Read
            # dict-compatibly so a plain-dict config (tests) and the typed AppConfig both work.
            # NOTE: the runner previously dropped these ? reels shipped UN-watermarked with the
            # ctor default paddings; this threads config.json through (the on-by-default mark).
            st = cfg.get("stitching") or {}
            VideoCompiler(
                local_work_dir,
                begin_padding=float(st.get("begin_padding", 0.5)),
                end_padding=float(st.get("end_padding", 0.5)),
                watermark=st.get("watermark"),
            ).compile_highlights(stitch_src, pairs, reel_name)
            _atomic_publish(local_reel, dest_reel)
            try:
                os.remove(local_reel)
            except OSError:
                pass
            logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
```

38. assistant (2026-06-17T14:33:40.365Z)

Let me get the full diff:

39. user (2026-06-17T14:33:41.663Z)

```
diff --git a/tools/generate_highlights.py b/tools/generate_highlights.py
index ae9f045..565663a 100644
--- a/tools/generate_highlights.py
+++ b/tools/generate_highlights.py
@@ -63,7 +63,8 @@ def _atomic_publish(local_path: str, dest_path: str) -> None:

 def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                               local_work_dir: str, db_path: str, force: bool = False,
-                              skip_ai_handoff: bool = False, wasb_stream: bool = False) ->
dict:
+                              skip_ai_handoff: bool = False, wasb_stream: bool = False,
+                              detect_from_paths: Optional[List[str]] = None) -> dict:
     """Generate one highlight reel per input video into out_dir. Returns a summary dict.

     Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
@@ -71,6 +72,11 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
     skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini handoff);
         candidate windows become the rallies. wasb_stream: have the WASB detector decode the
         video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).
+    detect_from_paths: optional list parallel to video_paths. When given, the detector runs on
+        detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source) but the
+        reel is stitched from video_paths[i] at full resolution. Rally timestamps transfer 1:1
+        (a pure temporal-preserving downscale shares the timeline). None = detect+stitch the
+        same source (legacy behaviour).
     """
     os.makedirs(out_dir, exist_ok=True)
     os.makedirs(local_work_dir, exist_ok=True)
@@ -115,13 +121,17 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
         reel_name = f"{stem} - Highlights.mp4"
         dest_reel = os.path.join(out_dir, reel_name)
         local_reel = os.path.join(local_work_dir, reel_name)
+        # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
+        # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
+        detect_src = detect_from_paths[i - 1] if detect_from_paths else src

         if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
             logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
```

```
            summary["skipped"].append(name)
            continue
-       if not os.path.exists(src):
-           logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
+       if not os.path.exists(src) or not os.path.exists(detect_src):
+           logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
+                          i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
            summary["failed"].append(name)
            continue

@@ -132,23 +142,29 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
        staged_src: Optional[str] = None
        try:
            logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
+           # Localize the DETECTION source for WSL detectors (the WSL FS can't read Drive/UNC
+           # via /mnt). The stitch source (`src`) is read by ffmpeg directly during compile,
so
+           # only the detector input ever needs staging.
            if needs_local:
-               staged_src = os.path.join(local_work_dir, name)
-               if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
-                   logger.info("localizing source (%.0f MB)...", os.path.getsize(src) / 1024 /
1024)
-                   shutil.copy2(src, staged_src)
-               proc_src = staged_src
+               staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
+               if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
+                   logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
+                   shutil.copy2(detect_src, staged_src)
+               proc_detect = staged_src
            else:
-               proc_src = src
+               proc_detect = detect_src

-           vid = compute_video_id(proc_src)
+           vid = compute_video_id(proc_detect)
            db.clear_video_data(vid)          # idempotent: no cross-run accumulation
-           db.add_video(vid, proc_src, "processed", [])
+           db.add_video(vid, proc_detect, "processed", [])
+           if detect_src != src:
+               logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
+                           os.path.basename(detect_src), name)
            logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

            # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
            # main.py); cast keeps the type checker honest about that contract.
-           result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
+           result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_detect)
            if isinstance(result, Failure):
                logger.error("segmentation FAILED: %s", getattr(result, "message", result))
                summary["failed"].append(name)
@@ -162,8 +178,20 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
                summary["no_rallies"].append(name)
                continue

-           # Compile locally, then publish atomically into the destination.
-           VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
+           # Stitch source: the original `src` when detecting on a separate proxy (hi-res
```

```
reel);
+            # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
+            stitch_src = src if detect_from_paths else proc_detect
+            # Honor the configured branding watermark + segment paddings (stitching.*). Read
+            # dict-compatibly so a plain-dict config (tests) and the typed AppConfig both work.
+            # NOTE: the runner previously dropped these ? reels shipped UN-watermarked with the
+            # ctor default paddings; this threads config.json through (the on-by-default mark).
+            st = cfg.get("stitching") or {}
+            VideoCompiler(
+                local_work_dir,
+                begin_padding=float(st.get("begin_padding", 0.5)),
+                end_padding=float(st.get("end_padding", 0.5)),
+                watermark=st.get("watermark"),
+            ).compile_highlights(stitch_src, pairs, reel_name)
             _atomic_publish(local_reel, dest_reel)
             try:
                 os.remove(local_reel)
@@ -209,6 +237,21 @@ def _resolve_videos(args) -> List[str]:
     raise SystemExit("provide --videos-dir or one/more --video")


+def _resolve_detect_from(args, video_paths: List[str]) -> Optional[List[str]]:
+    """Resolve --detect-from into a list parallel to video_paths (or None if unused).
+
+    Requires explicit --video (not --videos-dir) and a 1:1 count match ? each detection
+    proxy maps to exactly one --video by position."""
+    if not getattr(args, "detect_from", None):
+        return None
+    if not args.video:
+        raise SystemExit("--detect-from requires explicit --video (not --videos-dir)")
+    if len(args.detect_from) != len(args.video):
+        raise SystemExit(f"--detect-from count ({len(args.detect_from)}) must match "
+                         f"--video count ({len(args.video)}); pair them 1:1 in order")
+    return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in
args.detect_from]
+
+
 def main() -> None:
     ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
     ap.add_argument("--videos-dir", help="folder of source videos (all videos in it, unless
--video given)")
@@ -225,10 +268,18 @@ def main() -> None:
     ap.add_argument("--wasb-stream", action="store_true",
                     help="WASB decode-on-the-fly fast path (#178; output-preserving), instead
of "
                          "dumping ~46k PNGs. Only affects wasb_hybrid.")
+    ap.add_argument("--detect-from", action="append",
+                    help="detection source ? e.g. a 1080p proxy. The detector analyzes THIS "
+                         "(cheaper to decode) while the reel is stitched from the matching
--video "
+                         "at full resolution (e.g. the 4K original). Repeat to pair 1:1 with
--video "
+                         "in the same order. Timestamps transfer 1:1 (pure temporal
downscale).")
     args = ap.parse_args()
-    generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
+    video_paths = _resolve_videos(args)
+    detect_from_paths = _resolve_detect_from(args, video_paths)
+    generate_highlights_batch(video_paths, args.out, segmenter=args.segmenter,
                               local_work_dir=args.local_work, db_path=args.db,
force=args.force,
-                              skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream)
```

```
+                               skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream,
+                               detect_from_paths=detect_from_paths)


 if __name__ == "__main__":
```

40. assistant (2026-06-17T14:33:42.977Z)

Now let me check the updated tests:

41. user (2026-06-17T14:33:44.136Z)

```
"""tools/generate_highlights.py ? the batch highlight runner. The load-bearing guarantees
are NON-DESTRUCTIVE + atomic publish + idempotent skip; those are what we lock here
(heavy deps ? segmenter/compiler/db ? are mocked)."""
import os
from unittest.mock import MagicMock, patch

import pytest

import tools.generate_highlights as gh


def _fake_compiler(captured):
    """A stand-in VideoCompiler that records its ctor kwargs + compile args and writes the
    expected reel file (so the runner's atomic-publish step succeeds)."""
    class FakeCompiler:
        def __init__(self, output_dir, end_padding=1.0, begin_padding=0.5, watermark=None):
            captured["ctor"] = dict(output_dir=output_dir, end_padding=end_padding,
                                    begin_padding=begin_padding, watermark=watermark)

        def compile_highlights(self, raw_video_path, segments, output_filename):
            captured["compile"] = dict(raw=raw_video_path, segments=list(segments),
                                       out=output_filename)
            reel = os.path.join(captured["ctor"]["output_dir"], output_filename)
            with open(reel, "wb") as fh:
                fh.write(b"reel-bytes")
            return reel

    return FakeCompiler


def test_atomic_publish_is_nondestructive(tmp_path):
    src = tmp_path / "local_reel.mp4"
    src.write_bytes(b"reel-bytes")
    dest_dir = tmp_path / "out"
    dest_dir.mkdir()
    sibling = dest_dir / "Someone Else - Highlights.mp4"
    sibling.write_bytes(b"do-not-touch")

    gh._atomic_publish(str(src), str(dest_dir / "Me - Highlights.mp4"))

    assert (dest_dir / "Me - Highlights.mp4").read_bytes() == b"reel-bytes"
    assert sibling.read_bytes() == b"do-not-touch"          # sibling untouched
    assert not list(dest_dir.glob("*.publishing.tmp"))      # no partial left behind


def test_resolve_videos_explicit_and_dir(tmp_path):
    (tmp_path / "a.MP4").write_bytes(b"x")
    (tmp_path / "b.mp4").write_bytes(b"x")
    (tmp_path / "notes.txt").write_text("ignore")
    args = MagicMock(video=None, videos_dir=str(tmp_path))
    found = gh._resolve_videos(args)
    assert [os.path.basename(p) for p in found] == ["a.MP4", "b.mp4"]   # videos only, sorted
```

```
    args2 = MagicMock(video=["only.MP4"], videos_dir=str(tmp_path))
    assert gh._resolve_videos(args2) == [os.path.join(str(tmp_path), "only.MP4")]


def test_batch_skips_existing_reel_and_preserves_other_work(tmp_path):
    """force=False: a video whose reel already exists is skipped, the segmenter is never
    called, and unrelated files already in the destination are left intact."""
    out = tmp_path / "out"
    out.mkdir()
    (out / "Adarsh v Avi - Highlights.mp4").write_bytes(b"already-published")
    (out / "Adarsh v Avi - run.log").write_text("prior log")
    other = out / "Someone Else - Highlights.mp4"
    other.write_bytes(b"untouched")
    src = tmp_path / "Adarsh v Avi.MP4"
    src.write_bytes(b"src")

    seg_registry = MagicMock()
    with patch.object(gh, "load_config", return_value={}), \
         patch.object(gh, "Database", return_value=MagicMock()), \
         patch.object(gh, "segmenter_registry", seg_registry):
        summary = gh.generate_highlights_batch(
            [str(src)], str(out), segmenter="gemini",
            local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
force=False)

    assert summary["skipped"] == ["Adarsh v Avi.MP4"]
    # never processed: the resolved segmenter class was never instantiated/run
    seg_registry.get.return_value.assert_not_called()
    assert other.read_bytes() == b"untouched"            # other work preserved
    assert (out / "Adarsh v Avi - Highlights.mp4").read_bytes() == b"already-published"  # not
regenerated


def test_batch_applies_local_overrides(tmp_path):
    """--skip-ai-handoff / --wasb-stream inject into the typed config before segmentation."""
    from backend.config import AppConfig
    cfg = AppConfig()
    assert cfg.indexing.skip_ai_handoff is False          # default off
    assert cfg.indexing.wasb.stream_video is False

    out = tmp_path / "out"
    out.mkdir()
    src = tmp_path / "clip.MP4"
    src.write_bytes(b"src")

    seg_instance = MagicMock()
    seg_instance.process_video.return_value = []          # no rallies -> no compile/publish
    seg_registry = MagicMock()
    seg_registry.get.return_value = MagicMock(return_value=seg_instance)

    with patch.object(gh, "load_config", return_value=cfg), \
         patch.object(gh, "Database", return_value=MagicMock()), \
         patch.object(gh, "compute_video_id", return_value="vid123"), \
         patch.object(gh, "segmenter_registry", seg_registry):
        gh.generate_highlights_batch(
            [str(src)], str(out), segmenter="gemini",
            local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
            force=True, skip_ai_handoff=True, wasb_stream=True)

    assert cfg.indexing.skip_ai_handoff is True
    assert cfg.indexing.wasb.stream_video is True


def _run_one(tmp_path, *, video_paths, detect_from_paths=None, segmenter="gemini"):
    """Drive generate_highlights_batch once with a fake compiler/segmenter/db, returning the
```

```
    captured VideoCompiler ctor+compile args and the compute_video_id mock."""
    from backend.config import AppConfig
    cfg = AppConfig()                                  # watermark on by default
    out = tmp_path / "out"
    out.mkdir()
    db = MagicMock()
    db.query_play_segments.return_value = [{"start_time": 1.0, "end_time": 5.0}]
    seg_instance = MagicMock()
    seg_instance.process_video.return_value = ["ok"]    # non-Failure
    seg_registry = MagicMock()
    seg_registry.get.return_value = MagicMock(return_value=seg_instance)
    captured: dict = {}
    cvid = MagicMock(return_value="vid123")
    with patch.object(gh, "load_config", return_value=cfg), \
         patch.object(gh, "Database", return_value=db), \
         patch.object(gh, "compute_video_id", cvid), \
         patch.object(gh, "VideoCompiler", _fake_compiler(captured)), \
         patch.object(gh, "segmenter_registry", seg_registry):
        gh.generate_highlights_batch(
            [str(p) for p in video_paths], str(out), segmenter=segmenter,
            local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
            force=True, detect_from_paths=[str(p) for p in detect_from_paths] if
detect_from_paths else None)
    return captured, cvid, out


def test_batch_threads_watermark_and_padding_into_compiler(tmp_path):
    """REGRESSION: the runner must pass stitching.watermark + begin/end_padding to VideoCompiler
    (it previously constructed VideoCompiler bare -> reels shipped un-watermarked)."""
    src = tmp_path / "clip.MP4"
    src.write_bytes(b"src")
    captured, _cvid, out = _run_one(tmp_path, video_paths=[src])

    wm = captured["ctor"]["watermark"]
    assert wm and wm.get("enabled") is True                # on-by-default mark threaded
through
    assert wm.get("text") == "Generated by Khelsutra.guru"
    assert captured["ctor"]["begin_padding"] == 0.5        # config paddings, not the ctor 1.0
default
    assert captured["ctor"]["end_padding"] == 0.5
    assert captured["compile"]["raw"] == str(src)          # stitches from the (only) source
    assert (out / "clip - Highlights.mp4").exists()        # published


def test_batch_detect_from_proxy_stitches_from_original(tmp_path):
    """--detect-from: the detector runs on the proxy (video_id keyed off it) but the reel is
    stitched from the full-res --video, and named after it."""
    orig = tmp_path / "GX01.MP4"
    orig.write_bytes(b"original-4k")
    proxy = tmp_path / "GX01_1080p.mp4"
    proxy.write_bytes(b"proxy-1080")
    captured, cvid, out = _run_one(tmp_path, video_paths=[orig], detect_from_paths=[proxy])

    cvid.assert_called_once_with(str(proxy))               # detection keyed off the proxy
    assert captured["compile"]["raw"] == str(orig)         # reel stitched from the original
    assert (out / "GX01 - Highlights.mp4").exists()        # named after the original, not the
proxy


def test_resolve_detect_from_validates_pairing(tmp_path):
    assert gh._resolve_detect_from(MagicMock(detect_from=None), ["v"]) is None
    with pytest.raises(SystemExit):                        # count mismatch
        gh._resolve_detect_from(MagicMock(detect_from=["a", "b"], video=["v1"],
videos_dir=None), ["v1"])
    with pytest.raises(SystemExit):                        # requires explicit --video
```

```
        gh._resolve_detect_from(MagicMock(detect_from=["a"], video=None, videos_dir="d"),
["v1"])
    abs = os.path.join(os.sep, "abs", "p.mp4")
    assert gh._resolve_detect_from(
        MagicMock(detect_from=[abs], video=["v1"], videos_dir=None), ["v1"]) == [abs]
```

42. assistant (2026-06-17T14:33:46.516Z)

Excellent! Now I have all the information I need. Let me analyze the code for the specific issues mentioned in the review task. Let me check the VideoCompiler signature to understand the parameters:

43. user (2026-06-17T14:33:47.357Z)

```
16:class VideoCompiler:
17:    def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float = 0.5,
```

44. assistant (2026-06-17T14:33:48.827Z)

Now let me verify the specific issues mentioned in the review task by analyzing the code thoroughly. Let me look at the enumerate and batch indexing:

45. user (2026-06-17T14:33:49.281Z)

```
112         for i, src in enumerate(video_paths, 1):
113             name = os.path.basename(src)
114             stem = os.path.splitext(name)[0]
115             reel_name = f"{stem} - Highlights.mp4"
116             dest_reel = os.path.join(out_dir, reel_name)
117             local_reel = os.path.join(local_work_dir, reel_name)
118 
119             if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
120                 logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
name)
121                 summary["skipped"].append(name)
122                 continue
123             if not os.path.exists(src):
124                 logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths),
name)
125                 summary["failed"].append(name)
126                 continue
127 
128             run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
129             run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
130             run_fh.setFormatter(fmt)
131             logging.getLogger().addHandler(run_fh)
132             staged_src: Optional[str] = None
133             try:
134                 logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
135                 if needs_local:
136                     staged_src = os.path.join(local_work_dir, name)
137                     if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
138                         logger.info("localizing source (%.0f MB)...", os.path.getsize(src) /
1024 / 1024)
139                         shutil.copy2(src, staged_src)
140                     proc_src = staged_src
141                 else:
142                     proc_src = src
143 
144                 vid = compute_video_id(proc_src)
145                 db.clear_video_data(vid)          # idempotent: no cross-run accumulation
146                 db.add_video(vid, proc_src, "processed", [])
147                 logger.info("video_id=%s segmenting (%s)...", vid, segmenter)
```

```
148
149                 # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
150                 # main.py); cast keeps the type checker honest about that contract.
151                 result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
152                 if isinstance(result, Failure):
153                     logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
154                     summary["failed"].append(name)
155                     continue
156
157                 segs = db.query_play_segments(vid, {})
158                 pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
159                 logger.info("rallies detected: %d", len(pairs))
160                 if not pairs:
161                     logger.warning("no rallies -> no reel")
```

46. assistant (2026-06-17T14:33:52.212Z)

Wait, I'm looking at the old version in the working directory. Let me re-examine the file in the current branch by reading the full current version carefully:

47. user (2026-06-17T14:33:52.902Z)

```
"""Batch highlight-reel runner ? orchestrates "generate highlights for these videos
into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.

Why this exists (lessons paid for the hard way):
- **Never delete other work.** Reels + per-video run.logs are written to a LOCAL work dir
  first, then COPIED into the destination (atomically: temp name -> os.replace). The
  destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a
  Google-Drive-synced folder once lost finished reels + logs.)
- **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync
  client (deleted files resurrect, partial writes sync). Generate locally; publish once.
- **Idempotent re-runs.** Each video's prior DB segments are cleared before processing
  (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates
  duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
- **Resumable & concurrency-friendly.** A video whose reel already exists in the
  destination is skipped (unless --force); launching a second request never undoes the
  first's published reels. The Gemini path also isolates its chunk workdir per video
  (PR for output/chunks/<video_id>) so concurrent videos don't collide.

Usage:
  python -m tools.generate_highlights --videos-dir "G:\\...\\June 12 2026" \
      --out "G:\\...\\June 12 2026\\Highlights" --segmenter gemini [--force]
  python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
"""
from __future__ import annotations

import argparse
import logging
import os
import shutil
import sys
import traceback
from typing import Any, Dict, List, Optional, cast

from dotenv import load_dotenv

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from backend.config import load_config
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.results import Failure
```

```
from backend.utils.hashing import compute_video_id

logger = logging.getLogger("highlights_runner")
```

Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.

```
_WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
_VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")


def _atomic_publish(local_path: str, dest_path: str) -> None:
    """Copy local_path -> dest_path so the destination never sees a partial file.

    Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
    anything else in the destination folder.
    """
    os.makedirs(os.path.dirname(dest_path), exist_ok=True)
    tmp = dest_path + ".publishing.tmp"
    shutil.copy2(local_path, tmp)
    os.replace(tmp, dest_path)


def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                              local_work_dir: str, db_path: str, force: bool = False,
                              skip_ai_handoff: bool = False, wasb_stream: bool = False,
                              detect_from_paths: Optional[List[str]] = None) -> dict:
    """Generate one highlight reel per input video into out_dir. Returns a summary dict.

    Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.

    skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini handoff);
        candidate windows become the rallies. wasb_stream: have the WASB detector decode the
        video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).
    detect_from_paths: optional list parallel to video_paths. When given, the detector runs on
        detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source) but the
        reel is stitched from video_paths[i] at full resolution. Rally timestamps transfer 1:1
        (a pure temporal-preserving downscale shares the timeline). None = detect+stitch the
        same source (legacy behaviour).
    """
    os.makedirs(out_dir, exist_ok=True)
    os.makedirs(local_work_dir, exist_ok=True)
    needs_local = segmenter in _WSL_DETECTORS

    # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is invoked
    # standalone ? backend.main does this, but this module is its own entrypoint. Without it the
    # AI segmenter finds no API key and silently produces zero rallies. The fully-local path
    # (skip_ai_handoff) needs no key, so a missing .env is harmless there.
    load_dotenv()

    cfg = load_config()
    # Per-run overrides (typed config; the segmenter reads these via its dict-compat .get).
    # Logged by the segmenter itself once logging is configured below.
    if skip_ai_handoff:
        cfg.indexing.skip_ai_handoff = True
    if wasb_stream:
        cfg.indexing.wasb.stream_video = True
    db = Database(db_path)
    seg_cls = segmenter_registry.get(segmenter)
    if seg_cls is None:
        raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")

    # Combined batch log (local; published at the end).
    fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s", "%H:%M:%S")
    if not logging.getLogger().handlers:
```

```
        logging.basicConfig(level=logging.INFO, handlers=[logging.StreamHandler(sys.stdout)])
        for h in logging.getLogger().handlers:
            h.setFormatter(fmt)
    batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
    batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
    batch_fh.setFormatter(fmt)
    logging.getLogger().addHandler(batch_fh)

    summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [], "failed": []}
    logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
                len(video_paths), out_dir, segmenter, force)

    for i, src in enumerate(video_paths, 1):
        name = os.path.basename(src)
        stem = os.path.splitext(name)[0]
        reel_name = f"{stem} - Highlights.mp4"
        dest_reel = os.path.join(out_dir, reel_name)
        local_reel = os.path.join(local_work_dir, reel_name)
        # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
        # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
        detect_src = detect_from_paths[i - 1] if detect_from_paths else src

        if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
            logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
            summary["skipped"].append(name)
            continue
        if not os.path.exists(src) or not os.path.exists(detect_src):
            logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
                           i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
            summary["failed"].append(name)
            continue

        run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
        run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
        run_fh.setFormatter(fmt)
        logging.getLogger().addHandler(run_fh)
        staged_src: Optional[str] = None
        try:
            logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
            # Localize the DETECTION source for WSL detectors (the WSL FS can't read Drive/UNC
            # via /mnt). The stitch source (`src`) is read by ffmpeg directly during compile, so
            # only the detector input ever needs staging.
            if needs_local:
                staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
                if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
                    logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
                    shutil.copy2(detect_src, staged_src)
                proc_detect = staged_src
            else:
                proc_detect = detect_src

            vid = compute_video_id(proc_detect)
            db.clear_video_data(vid)          # idempotent: no cross-run accumulation
            db.add_video(vid, proc_detect, "processed", [])
            if detect_src != src:
                logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
                            os.path.basename(detect_src), name)
            logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

            # AppConfig is consumed by segmenters via its dict-compat .get shim (same as
            # main.py); cast keeps the type checker honest about that contract.
```

```
            result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_detect)
            if isinstance(result, Failure):
                logger.error("segmentation FAILED: %s", getattr(result, "message", result))
                summary["failed"].append(name)
                continue

            segs = db.query_play_segments(vid, {})
            pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
            logger.info("rallies detected: %d", len(pairs))
            if not pairs:
                logger.warning("no rallies -> no reel")
                summary["no_rallies"].append(name)
                continue

            # Stitch source: the original `src` when detecting on a separate proxy (hi-res
reel);
            # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
            stitch_src = src if detect_from_paths else proc_detect
            # Honor the configured branding watermark + segment paddings (stitching.*). Read
            # dict-compatibly so a plain-dict config (tests) and the typed AppConfig both work.
            # NOTE: the runner previously dropped these ? reels shipped UN-watermarked with the
            # ctor default paddings; this threads config.json through (the on-by-default mark).
            st = cfg.get("stitching") or {}
            VideoCompiler(
                local_work_dir,
                begin_padding=float(st.get("begin_padding", 0.5)),
                end_padding=float(st.get("end_padding", 0.5)),
                watermark=st.get("watermark"),
            ).compile_highlights(stitch_src, pairs, reel_name)
            _atomic_publish(local_reel, dest_reel)
            try:
                os.remove(local_reel)
            except OSError:
                pass
            logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
            summary["done"].append(name)
        except Exception:
            logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
            summary["failed"].append(name)
        finally:
            logging.getLogger().removeHandler(run_fh)
            run_fh.close()
            # publish this video's run.log (additive; never deletes anything in out_dir)
            try:
                _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} - run.log"))
            except OSError:
                pass
            if staged_src and os.path.exists(staged_src):
                try:
                    os.remove(staged_src)
                except OSError:
                    pass

    logging.getLogger().removeHandler(batch_fh)
    batch_fh.close()
    try:
        _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
    except OSError:
        pass
    logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
    return summary


def _resolve_videos(args) -> List[str]:
```

```
    if args.video:
        return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in
args.video]
    if args.videos_dir:
        return sorted(
            os.path.join(args.videos_dir, f) for f in os.listdir(args.videos_dir)
            if f.endswith(_VIDEO_EXTS) and os.path.isfile(os.path.join(args.videos_dir, f))
        )
    raise SystemExit("provide --videos-dir or one/more --video")


def _resolve_detect_from(args, video_paths: List[str]) -> Optional[List[str]]:
    """Resolve --detect-from into a list parallel to video_paths (or None if unused).

    Requires explicit --video (not --videos-dir) and a 1:1 count match ? each detection
    proxy maps to exactly one --video by position."""
    if not getattr(args, "detect_from", None):
        return None
    if not args.video:
        raise SystemExit("--detect-from requires explicit --video (not --videos-dir)")
    if len(args.detect_from) != len(args.video):
        raise SystemExit(f"--detect-from count ({len(args.detect_from)}) must match "
                         f"--video count ({len(args.video)}); pair them 1:1 in order")
    return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in
args.detect_from]


def main() -> None:
    ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
    ap.add_argument("--videos-dir", help="folder of source videos (all videos in it, unless
--video given)")
    ap.add_argument("--video", action="append", help="explicit source video (repeatable)")
    ap.add_argument("--out", required=True, help="destination folder for reels + run.logs")
    ap.add_argument("--segmenter", default="gemini", help="segmenter registry key (default:
gemini)")
    ap.add_argument("--local-work", default=os.path.join("output", "highlights_work"),
                    help="local staging dir (reels compiled here, then published)")
    ap.add_argument("--db", default=os.path.join("output", "sports_indexer.db"))
    ap.add_argument("--force", action="store_true", help="regenerate even if a reel already
exists")
    ap.add_argument("--skip-ai-handoff", action="store_true",
                    help="FULLY-LOCAL: trajectory-hybrid segmenters emit candidate windows as "
                         "rallies with NO Gemini handoff (no API key / upload). Ignored by
--segmenter gemini.")
    ap.add_argument("--wasb-stream", action="store_true",
                    help="WASB decode-on-the-fly fast path (#178; output-preserving), instead of
"
                         "dumping ~46k PNGs. Only affects wasb_hybrid.")
    ap.add_argument("--detect-from", action="append",
                    help="detection source ? e.g. a 1080p proxy. The detector analyzes THIS "
                         "(cheaper to decode) while the reel is stitched from the matching
--video "
                         "at full resolution (e.g. the 4K original). Repeat to pair 1:1 with
--video "
                         "in the same order. Timestamps transfer 1:1 (pure temporal
downscale).")
    args = ap.parse_args()
    video_paths = _resolve_videos(args)
    detect_from_paths = _resolve_detect_from(args, video_paths)
    generate_highlights_batch(video_paths, args.out, segmenter=args.segmenter,
                              local_work_dir=args.local_work, db_path=args.db, force=args.force,
                              skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream,
                              detect_from_paths=detect_from_paths)
```

```
if __name__ == "__main__":
    main()
```

48. assistant (2026-06-17T14:33:54.623Z)

Perfect! Now I can conduct a thorough code review. Let me analyze the implementation against the review criteria:

49. user (2026-06-17T14:33:55.175Z)

Chapter marked: "Code Review Analysis". Continue your current work.

50. assistant (2026-06-17T14:34:17.828Z)

Now let me systematically review each criterion:

Analysis of `generate_highlights.py` - Proxy Detection Feature Review

1. **Enumerate Batch Index Pairing (Line 120 & Line 126) ?**
- **Line 120**: `for i, src in enumerate(video_paths, 1):` starts at 1
- **Line 126**: `detect_src = detect_from_paths[i - 1] if detect_from_paths else src`
- **Finding**: Correct pairing! The 1-indexed loop position `i` is properly mapped back to 0-indexed list access with `i - 1`.
- **Tests**: Line 238 in tests verifies this with 1:1 pairing via `_resolve_detect_from`.

2. **Detection Runs on Proxy (Video_id keyed off detect_src) ?**
- **Line 161**: `vid = compute_video_id(proc_detect)` - detection is keyed off the proxy (proc_detect, not src)
- **Line 162-163**: `db.add_video(vid, proc_detect, ...)` - database registers the proxy as the input
- **Line 172**: `result = seg_cls(db, ...).process_video(vid, proc_detect)` - segmentation runs on proxy
- **Tests**: Line 211 validates `cvid.assert_called_once_with(str(proxy))`
- **Finding**: Correct implementation ?

3. **Reel Stitched from Full-Res Video (--video) ?**
- **Line 180**: `stitch_src = src if detect_from_paths else proc_detect`
- **When detect_from_paths is set**: stitch_src = src (original 4K)
- **When detect_from_paths is None**: stitch_src = proc_detect (legacy behavior)
- **Line 186-192**: VideoCompiler called with `stitch_src` (full res)
- **Tests**: Line 211 validates `captured["compile"]["raw"] == str(orig)`
- **Finding**: Correct implementation ?

4. **Reel Named After --video (Original Source) ?**
- **Line 113**: `name = os.path.basename(src)` - always based on the stitch source (src)
- **Line 114**: `stem = os.path.splitext(name)[0]` - uses src's filename
- **Line 115**: `reel_name = f"{stem} - Highlights.mp4"` - reel named after src
- **Tests**: Line 212 validates `(out / "GX01 - Highlights.mp4").exists()` (named after original, not proxy)
- **Finding**: Correct implementation ?

5. **_resolve_detect_from Validation ???**
- **Line 223-237**: Three critical validations:
  1. **1:1 Count Match**: Line 233 `if len(args.detect_from) != len(args.video)`
  2. **Explicit --video Required**: Line 231 `if not args.video` raises SystemExit
  3. **Path Resolution**: Line 236 properly resolves relative paths like --video does
- **Tests**: Lines 215-227 test all three cases with pytest.raises(SystemExit)
- **Finding**: Robust validation ?

6. **Missing-Source Check Covers Both src and detect_src ?**
- **Line 133**: `if not os.path.exists(src) or not os.path.exists(detect_src):`
- **Line 134-135**: Detailed logging shows both: `(stitch_exists=%s detect_exists=%s)` with values

- **Edge case**: If detect_src is a proxy for src and src is 4K, both must be present (correct)
- **Finding**: Comprehensive check ?

7. **Staging/Localization for WSL Detectors ?**
- **Line 142-145**: Correctly localizes `detect_src` (not src) for WSL detectors
  ```python
  staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
  shutil.copy2(detect_src, staged_src)  # copies the PROXY for WSL
  proc_detect = staged_src
  ```
- **Comment lines 138-140**: Explicitly clarifies the distinction
- **Finding**: Correct isolation of detection source ?

8. **VideoCompiler Invocation with Config Threading ?**
- **Line 176-185**: Properly threads stitching config (watermark, padding)
- **Line 177**: `st = cfg.get("stitching") or {}` - dict-compatible read for both typed and plain configs
- **Line 179-183**: Passes `begin_padding`, `end_padding`, `watermark` to ctor
- **Tests**: Lines 195-206 validate watermark/padding are honored
- **Finding**: Correct threading of config ?

EDGE CASES & POTENTIAL ISSUES

Edge Case 1: Staged-File Basename Collision Across Batch
Risk: When `--detect-from` provides multiple proxies with **same basename** but different paths
- Example: `/path1/clip_1080p.mp4` and `/path2/clip_1080p.mp4`
- **Line 145**: `os.path.join(local_work_dir, os.path.basename(detect_src))`
- **Issue**: Both would stage to `local_work_dir/clip_1080p.mp4`, clobbering the first
- **Severity**: MAJOR - Would silently corrupt the batch run
- **Requires**: Needs unique basename generation or subdirs per video

Edge Case 2: --detect-from Given But Path Missing (Validation Gap)
Risk: What if a --detect-from path is provided but doesn't exist until after _resolve_detect_from?
- **Line 223**: `_resolve_detect_from` only validates count and --video requirement
- **Lines 133-135**: Existence check happens per-loop but logs both src/detect_src as missing
- **Behavior**: Skip with warning (expected, not a bug)
- **Severity**: MINOR - Expected behavior, well-logged

Edge Case 3: Interaction with --skip-ai-handoff
Risk: --skip-ai-handoff + --detect-from together
- **Line 89-90**: skip_ai_handoff sets config override
- **Line 161-172**: Detection still runs on proxy (correct)
- **Line 172**: Result is processed regardless of skip_ai_handoff setting
- **Finding**: No interaction issue, both features compose independently ?

Edge Case 4: Dict Config Type Flexibility
Risk: `cfg.get("stitching")` assumes cfg is dict-compatible
- **Line 177**: `st = cfg.get("stitching") or {}`
- **AppConfig from load_config()**: Typed Pydantic model with .get() support
- **Tests line 8**: _run_one() creates AppConfig which supports .get()
- **Tests line 50-54**: Mocked as plain dict `{}` which also has .get()
- **Finding**: Properly handles both ?

TEST COVERAGE VERIFICATION

All critical paths tested:
- ? test_batch_detect_from_proxy_stitches_from_original (lines 208-212)

```
- ? test_batch_threads_watermark_and_padding_into_compiler (lines 195-206)
- ? test_resolve_detect_from_validates_pairing (lines 215-227)
- ? test_atomic_publish_is_nondestructive (lines 10-23)
- ? test_batch_skips_existing_reel_and_preserves_other_work (lines 36-61)

Missing test:
- **No test for basename collision edge case** (Edge Case 1)

---
```

SUMMARY OF FINDINGS

```
| Item | Result | Severity |
|------|--------|----------|
| Detection on proxy (detect_src keyed) | ? PASS | - |
| Reel from full-res (stitch_src) | ? PASS | - |
| Reel named after original --video | ? PASS | - |
| i-1 batch index pairing (1-indexed loop) | ? PASS | - |
| _resolve_detect_from validation (count, explicit --video) | ? PASS | - |
| Missing-source check (both src & detect_src) | ? PASS | - |
| Staged-file basename collision risk | ? FAIL | **MAJOR** |
| --detect-from + --skip-ai-handoff interaction | ? PASS | - |
| Config threading (watermark/padding) | ? PASS | - |
| WSL detector localization (detects proxy, not src) | ? PASS | - |
```

51. user (2026-06-17T14:34:22.786Z)

```
Structured output provided successfully
```